



中山大學
SUN YAT-SEN UNIVERSITY

本科生实验报告

实验课程: 操作系统原理实验

任课教师: 刘宁

实验题目: 编译内核利用已有内核构建 OS

专业名称: 计算机科学与技术

学生姓名: 林宏宇

学生学号: 23320093

实验地点: 实验中心 D501

实验时间: 20250226

Section 1 实验概述

熟悉 Linux 内核的编译过程和启动过程，并在自行编译内核的基础上构建简单应用并启动，同时能够利用精简的 BusyBox 构建简单的 OS，熟悉 Linux 的构建过程。

Section 2 预备知识与实验环境

- 预备知识：x86 汇编语言程序设计、Linux 系统命令行工具
- 实验环境：
 - 虚拟机版本/处理器型号：Virtualbox
 - 代码编辑环境：Vscode+nasm+C/C++插件
 - 代码编译工具：gcc/g++（64 位）
 - 重要三方库信息：

Section 3 实验任务

- 实验任务 1：环境配置
- 实验任务 2：编译 Linux 内核
- 实验任务 3：启动内核并调试
- 实验任务 4：制作 Initramfs
- 实验任务 5：加载 Initramfs 并再次使用 gdb 调试
- 实验任务 6：编译并启动 Busybox

Section 4 实验步骤与实验结果

----- 实验任务 1 -----

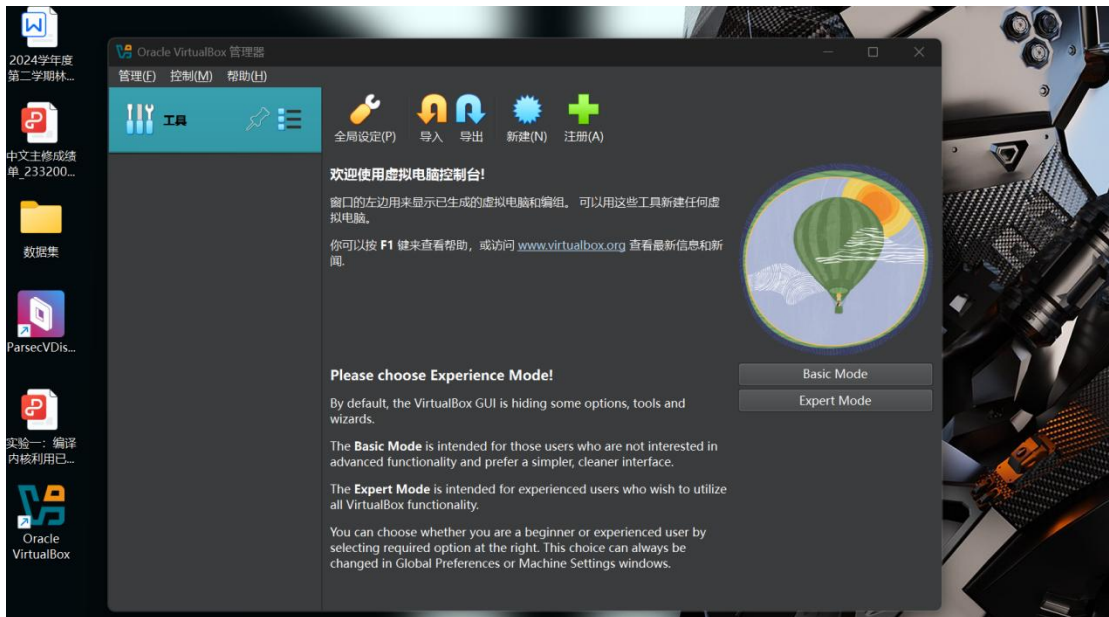
- 任务要求：配置环境
- 思路分析：

方法：先下载 Virtualbox 启动虚拟机，下载 ubuntu，配置 linux 环境并安装相应文件。

注意点：下载时严格按照实验步骤进行，防止出错。同时注意 VirtualBox 和 Ubuntu 的版本号，选择好相应路径进行下载。下载路径建议放在 C 盘，同时保留足够的内存空间，第一遍安装在 D 盘时启动失败。配置环境时，有时有权限限制，需要开放权限使用密码验证进行下载。

- 实验步骤:

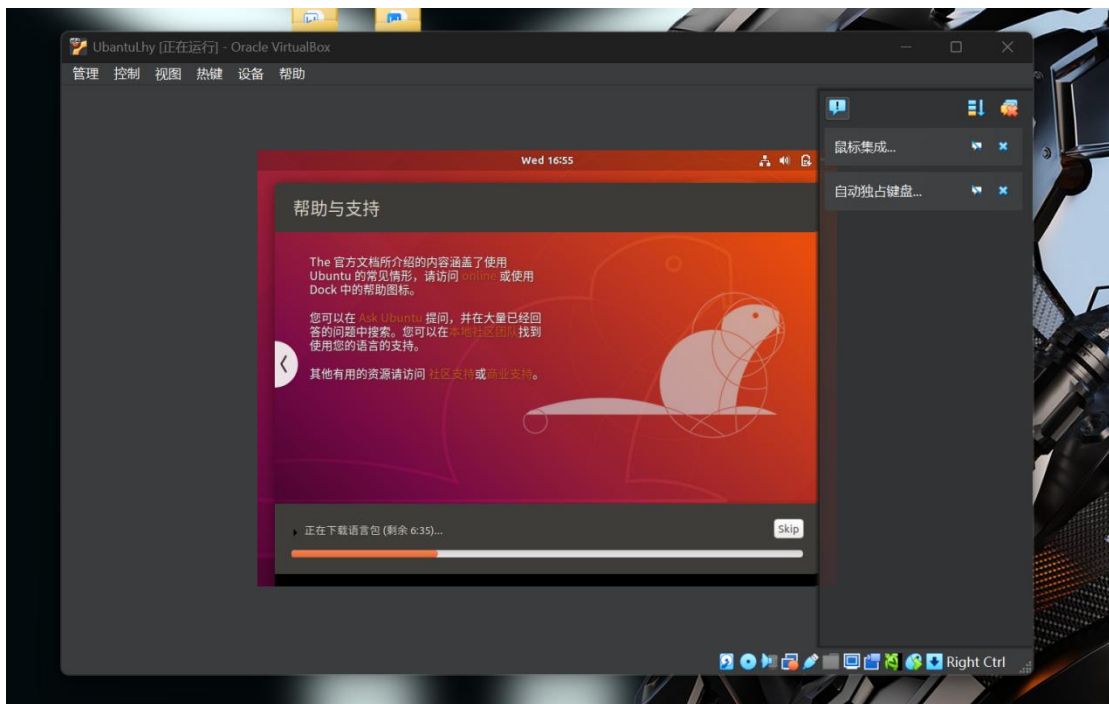
1. 先在官网下载 Virtualbox，路径放在 D 盘时出错，建议 C 盘



2. 根据 QQ 群链接下载 18.04 版本 ubuntu

virtualbox-7.1.6-167084-win	2025/2/26 15:45	应用程序	120,154 KB
ubuntu-18.04.6-desktop-amd64	2025/2/26 15:41	光盘映像文件	2,455,200 KB
实验一: 编译内核利用已有内核构建OS	2025/2/26 14:41	WPS PDF 文档	87 KB

3. 在 Virtualbox 新建，给虚拟机命名和配置 ubuntu 的虚拟光盘，启动下载



4. 配置 C/C++环境并安装软件

安装 gcc 环境

```
sudo apt install binutils
sudo apt install gcc
```

安装其他工具

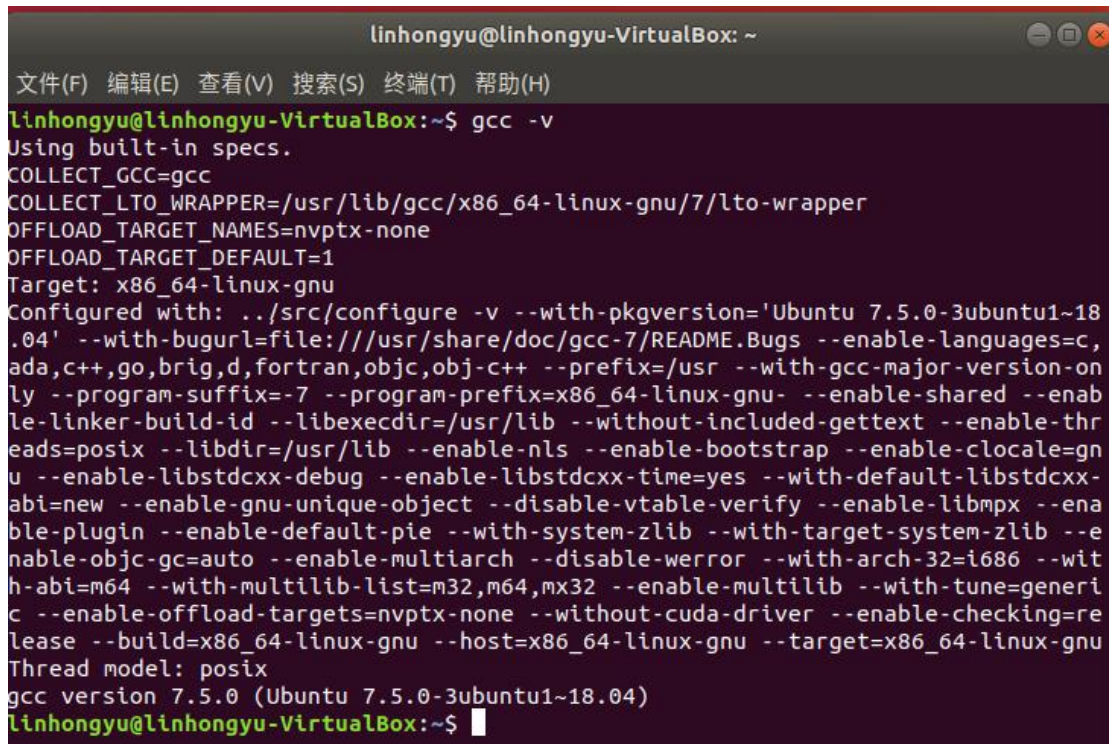
```
sudo apt install qemu
sudo apt install cmake
sudo apt install libncurses5-dev
sudo apt install bison
sudo apt install flex
sudo apt install libssl-dev
sudo apt install libc6-dev-i386
sudo apt install gcc-multilib
sudo apt install g++-multilib
```

安装 nasm，安装时需要权限，使用 sudo 命令输入密码即可下载

```
wget https://www.nasm.us/pub/nasm/releasebuilds/2.15/nasm-2.15.tar.gz
cd ./nasm-2.15
tar -vxf nasm-2.15.tar.gz
./configure
make install
sudo make install
```

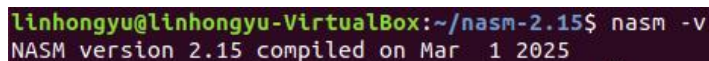
● 实验结果展示：

1. 检查 gcc 安装结果如下图，输出 gcc 的版本号表明安装成功

A terminal window titled 'linhongyu@linhongyu-VirtualBox: ~' with a menu bar (文件(F), 编辑(E), 查看(V), 搜索(S), 终端(T), 帮助(H)). The terminal shows the command 'gcc -v' and its output. The output includes configuration details for gcc 7.5.0 on Ubuntu 18.04, such as 'Using built-in specs.', 'COLLECT_GCC=gcc', 'Target: x86_64-linux-gnu', and 'gcc version 7.5.0 (Ubuntu 7.5.0-3ubuntu1~18.04)'. The prompt returns to 'linhongyu@linhongyu-VirtualBox:~\$' after the command.

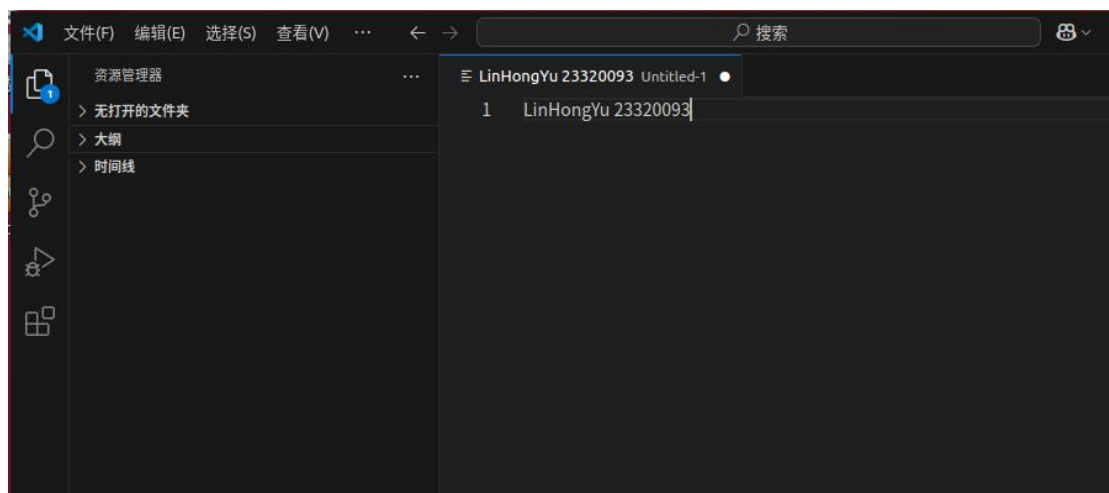
```
linhongyu@linhongyu-VirtualBox: ~
文件(F) 编辑(E) 查看(V) 搜索(S) 终端(T) 帮助(H)
linhongyu@linhongyu-VirtualBox:~$ gcc -v
Using built-in specs.
COLLECT_GCC=gcc
COLLECT_LTO_WRAPPER=/usr/lib/gcc/x86_64-linux-gnu/7/lto-wrapper
OFFLOAD_TARGET_NAMES=nvptx-none
OFFLOAD_TARGET_DEFAULT=1
Target: x86_64-linux-gnu
Configured with: ../src/configure -v --with-pkgversion='Ubuntu 7.5.0-3ubuntu1~18.04' --with-bugurl=file:///usr/share/doc/gcc-7/README.Bugs --enable-languages=c,ada,c++,go,brig,d,fortran,objc,obj-c++ --prefix=/usr --with-gcc-major-version-only --program-suffix=-7 --program-prefix=x86_64-linux-gnu- --enable-shared --enable-linker-build-id --libexecdir=/usr/lib --without-included-gettext --enable-threads=posix --libdir=/usr/lib --enable-nls --enable-bootstrap --enable-clocale=gnu --enable-libstdcxx-debug --enable-libstdcxx-time=yes --with-default-libstdcxx-abi=new --enable-gnu-unique-object --disable-vtable-verify --enable-libmpx --enable-plugin --enable-default-pie --with-system-zlib --with-target-system-zlib --enable-objc-gc=auto --enable-multiarch --disable-werror --with-arch-32=i686 --with-abi=m64 --with-multilib-list=m32,m64,mx32 --enable-multilib --with-tune=generic --enable-offload-targets=nvptx-none --without-cuda-driver --enable-checking=release --build=x86_64-linux-gnu --host=x86_64-linux-gnu --target=x86_64-linux-gnu
Thread model: posix
gcc version 7.5.0 (Ubuntu 7.5.0-3ubuntu1~18.04)
linhongyu@linhongyu-VirtualBox:~$
```

2. nasm 等安装结果

A terminal window showing the command 'nasm -v' and its output. The output is 'NASM version 2.15 compiled on Mar 1 2025'.

```
linhongyu@linhongyu-VirtualBox:~/nasm-2.15$ nasm -v
NASM version 2.15 compiled on Mar 1 2025
```

3. vscode 安装结果



----- 实验任务 2 -----

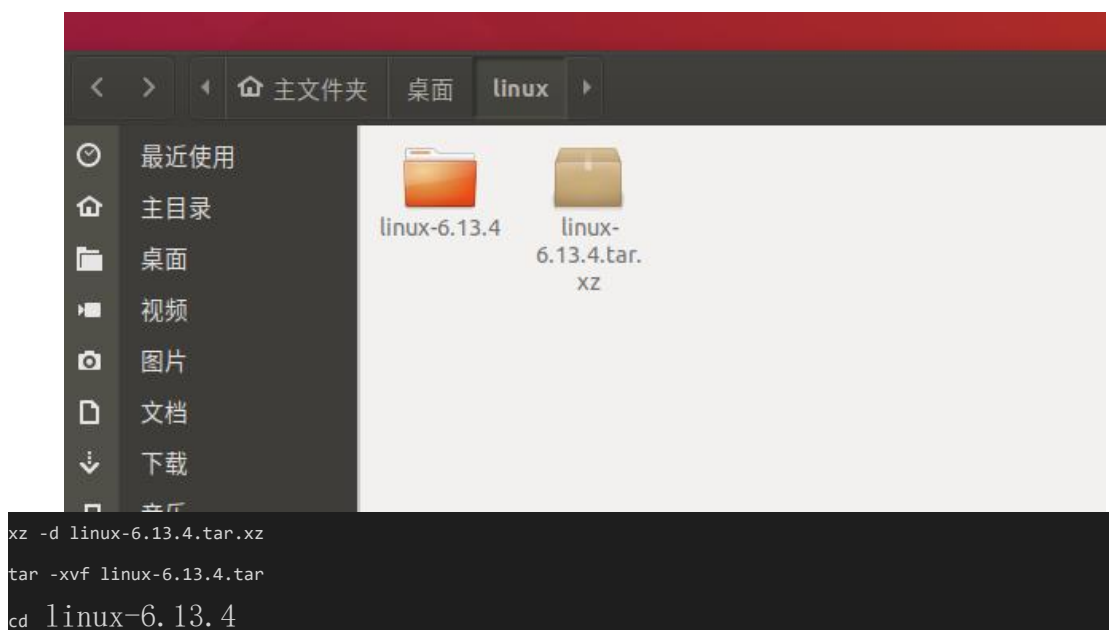
- 任务要求：编译 Linux 内核
- 思路分析：

方法：下载内核，并编译内核，熟悉终端命令行的使用。

注意点：下载、解压、创建文件夹等注意命令行的实现路径，有的命令需要在文件夹之下实现，不然会出现找不到文件或目录等步骤出错影响后续的进程。

- 实验步骤：

1. 从官网下载内核、解压、进入

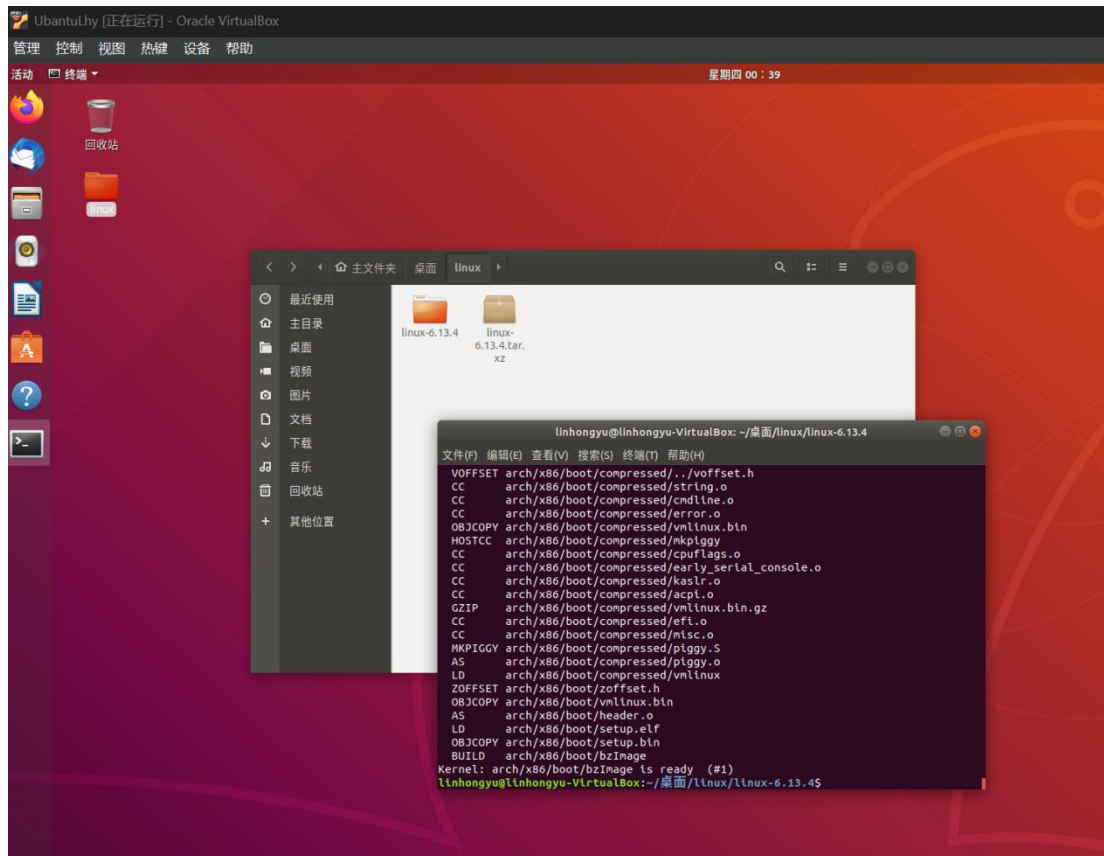


2. 编译内核

```
make i386_defconfig
make menuconfig
make -j8
```

- 实验结果展示:

安装完成, 可以看到 bzImage、vmlinuz 下载完成



----- 实验任务 3 -----

- 任务要求: 启动内核并调试
- 思路分析:

方法: 在一个终端启动内核, 再另一个终端远程调试。

注意点: 要使用两个终端分别输入命令行, 不是在同一个终端中进行, 同时这两个终端对应的路径都是 lab1 文件下。

- 实验步骤:

1. 启动内核

```
cd ~/lab1
qemu-system-i386 -kernel linux-6.13.4/arch/x86/boot/bzImage -s -S -append "console=ttyS0" -nographic
```

2. 调试

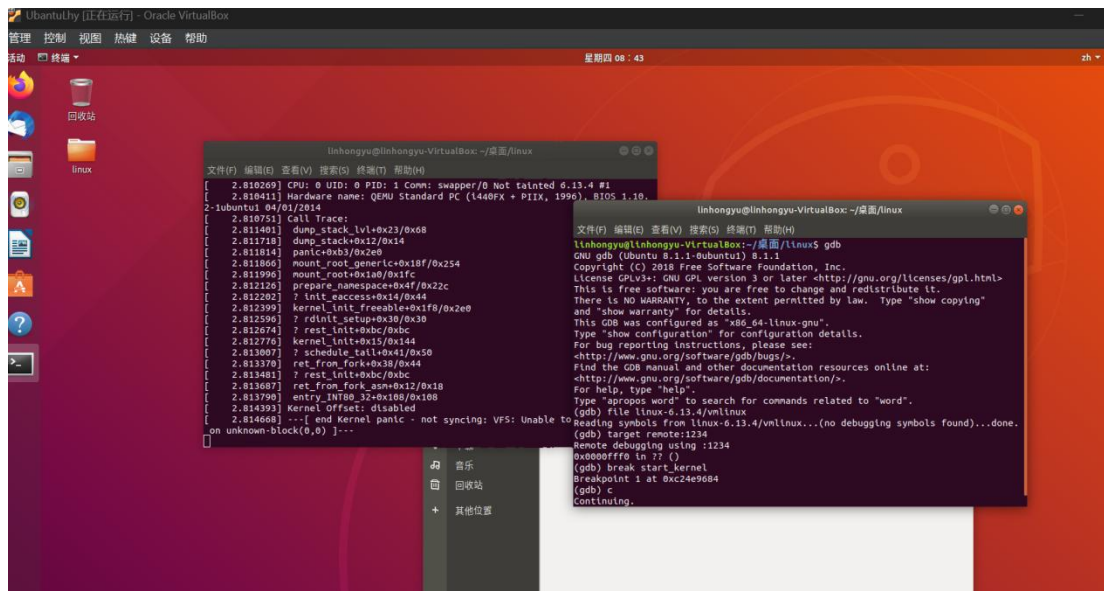
```
gdb
```



```
file linux-6.13.4/vmlinux
target remote:1234
break start_kernel
c
```

- 实验结果展示:

可以在终端中看到 en Kernal panic... 说明远程调试成功。



----- 实验任务 3 -----

- 任务要求: 制作 Initramfs

- 思路分析:

方法: 编写 hello world 程序, 加载 initramfs

注意点: 使用之前下载好的 vscode 编写 helloworld 代码, 在 lab1 中依次保存、生成可执行文件、打包、加载等操作, 再进行调试, 理解程序执行的过程。

- 实验步骤:

1. 编写 hello world.c 程序并保存

```
#include <stdio.h>

void main()
{
    printf("lab1: Hello World\n");
    fflush(stdout);
    while(1);
}
```

2. 编译 helloworld.c 可执行文件

```
gcc -o helloworld -m32 -static helloworld.c
```

3. 打包 initramfs、加载 initramfs

```
echo helloworld | cpio -o --format=newc > hwinitramfs
```

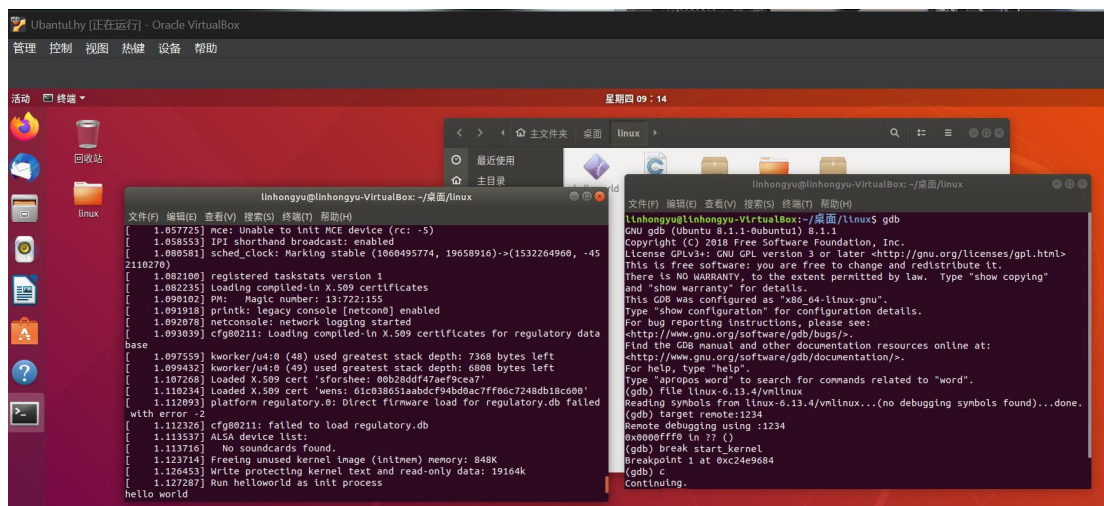
```
qemu-system-i386 -kernel linux-5.10.19/arch/x86/boot/bzImage -initrd hwinitramfs -s -S -append "console=ttyS0  
rdinit=helloworld" -nographic
```

4. gdb 远程调试, 重复之前的操作

```
gdb  
file linux-6.13.4/vmlinux  
target remote:1234  
break start_kernel  
c
```

● 实验结果展示:

可以看见 hello world 成功输出在终端中, 执行成功。



实验任务 4

● 任务要求: 编译并启动 Busybox

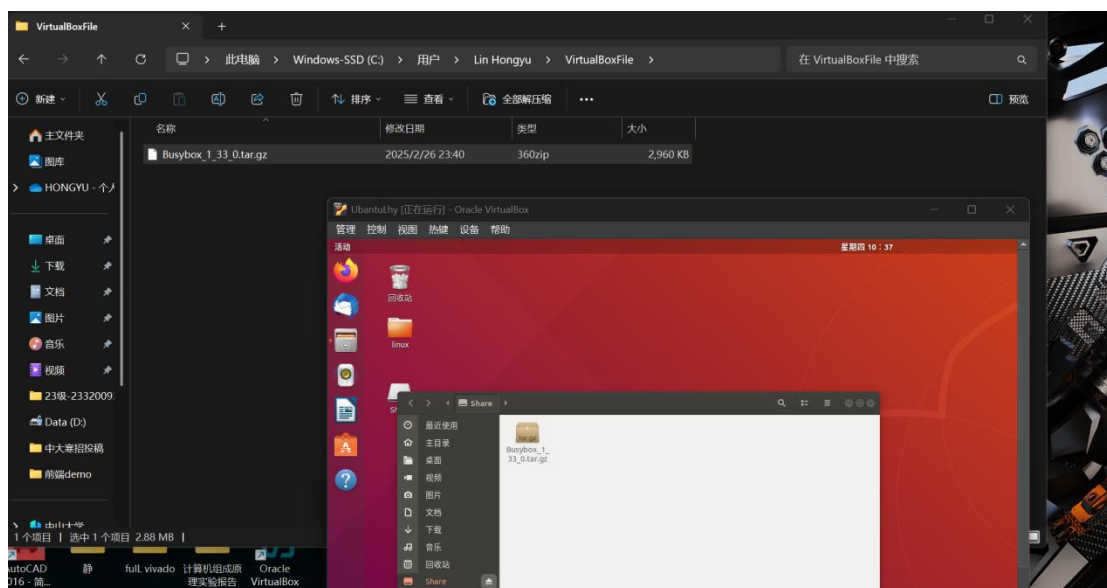
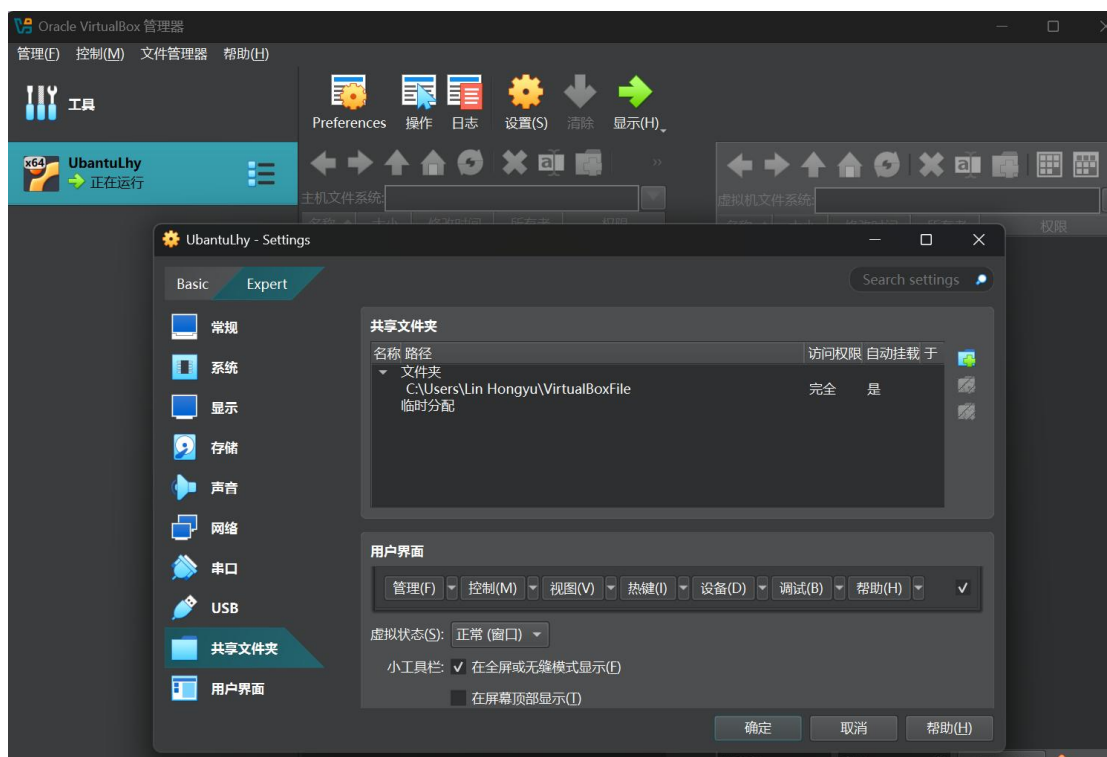
● 思路分析:

方法: 将 QQ 群的 busybox 安装包通过 VirtualBox 的共享文件传输, 从 Windows 传输到 Linux 中进行使用, 压缩、编译 busybox。再制作 Initramfs, 最后加载 busybox。

注意点: 制作 Initramfs 的步骤比较繁琐复杂, 容易出错, 严格按照实验流程, 区分在 lab1 下和在 mybusybox 的路径下实现命令行操作, 否则容易出错。

● 实验步骤:

1. 下载并解压，使用共享文件夹从 Windows 向 Linux 传输文件



3. 编译 busybox，完成设置

```
make defconfig
make menuconfig

make -j8
make install
```

4. 制作 initramfs

取出 install 目录下的文件放入 mybusybox

```
cd ~/lab1
mkdir mybusybox
mkdir -pv mybusybox/{bin,sbin,etc,proc,sys,usr/{bin,sbin}}
cp -av busybox-1_33_0/_install/* mybusybox/
cd mybusybox
```

编写 init 程序，脚本保存在 mybusybox 下，用命令 gredit 打开，脚本内容如下

```
mount -t proc none /proc
mount -t sysfs none /sys
echo -e "\nBoot took $(cut -d' ' -f1 /proc/uptime) seconds\n"
exec /bin/sh
```

需要添加执行权限

```
chmod u+x init
```

将 mybusybox 下文件打包，注意终端的路径是在 mybusybox 下，同时个人系统的路径不同，显示找不到目录和文件，需要添加~/桌面的路径。实验时通关观察到打包的块数量超大，发现打包了整个 lab1 导致出错

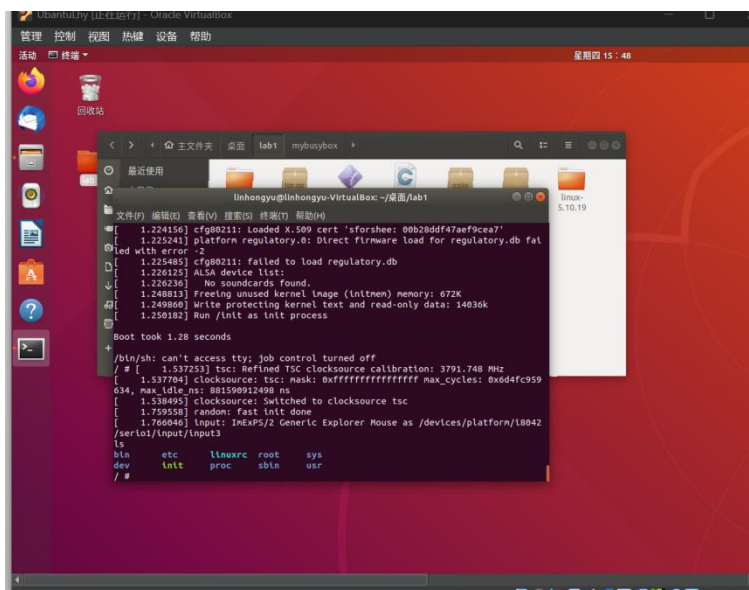
```
find . -print0 | cpio --null -ov --format=newc | gzip -9 > ~/桌面/lab1/initramfs-busybox-x86.cpio.gz
```

5. 加载 busybox

```
qemu-system-i386 -kernel linux-5.10.19/arch/x86/boot/bzImage -initrd initramfs-busybox-x86.cpio.gz -nographic
-append "console=ttyS0"
```

● 实验结果展示：

使用 ls 命令可以查看文件夹，加载 Busybox 成功



Section 5 实验总结与心得体会

1. 实验耗费时间精力较大的部分在于前期的下载和安装，对于 linux 不熟悉，并且系统会有些设置导致步骤失败等，需要上网查询相关解决办法，并且咨询助教才能解决，这也是新手必须经历的过程。

2. 前期的不熟悉，对 linux 的一些名词没有了解，导致执行命令时不知道应该在哪个路径下执行，也花费了不少时间。

3. 对于 linux 的系统有了一些了解，能够稍微入门并熟悉实验流程，能够阅读指导书和课件，也知道使用命令行进行操作，更是体验到了 linux 的魅力。

Section 6 对实验的改进建议和意见

1. 指导书内容不错，命令给的比较明确，一些操作也很细节，质量很高，但是前期的步骤对不了解 linux 的小白不是很友好，容易踩坑。

2. 指导书也许太久没更新，有些步骤给的不明确，对刚接触的新人不友好，还得同时根据 QQ 群进行对应步骤，如“环境配置”步骤，没有详细介绍 Virtualbox 的下载，直接就是讲换源，让刚入门的小白尚对 Linux 的操作不熟悉看不明白。

若在非Linux环境下，首先下载安装Virtualbox，用于启动虚拟机。如果本身是Linux环境则不需要这个步骤。建议安装Ubuntu 18.04桌面版，并将下载源换成清华下载源。

对于一个新安装的虚拟机，按如下步骤配置环境。

换源

由于ubuntu的下载源默认是国外的，为了提高下载速度，我们需要将下载源更换为国内源。我们首先备份原先的下载源。

```
sudo mv /etc/apt/sources.list /etc/apt/sources.list.backup
```

然后找到清华的ubuntu下载源

[<https://mirrors.tuna.tsinghua.edu.cn/help/ubuntu/>]。注意，选择你ubuntu的版本对应的下载源。

3. 有些指令给的不是很明确，在刚开始就“使用 gedit 打开”，可是没有了解 linux 的基础知识，怎么知道 gedit 是一个命令呢。

然后使用 gedit 打开下载源保存的文件 `/etc/apt/sources.list`

```
sudo gedit /etc/apt/sources.list
```

4. 有些命名并不准确，如这里的路径在我的 linux 下就会显示找不到目录或文件，需要自己修改路径，应当说明。

将x86-busybox下面的内容打包归档成cpio文件，以供Linux内核做initramfs启动执行。

```
find . -print0 | cpio --null -ov --format=newc | gzip -9 > ~/lab1
```

Section 7 附录：参考资料清单

- 1.https://blog.csdn.net/weixin_47195496/article/details/138069122
- 2.https://blog.csdn.net/weixin_42789301/article/details/108598098
- 3.https://blog.csdn.net/qq_33460264/article/details/118904489
- 4.https://blog.csdn.net/qq_42365082/article/details/127047885

Section 8 附录：代码清单

本节为可选章节，本次实验除了 hello world 没有代码，helloworld 代码在上面过程中已给出。